

WEEK13 설명문서

PhysX 기반 물리 시스템, Physics Asset Editor, Ragdoll, Vehicle, Depth of Field

대상 프로젝트: Jungle_Week13_Team2 / 범위: WEEK13 구현 파트

문서 목적	이번 주에 구현하려고 한 내용과 실제 코드 구현 구조를 설명한다.
분석 기준	WEEK13 과제 PDF와 업로드된 Team2 프로젝트 코드.
제외 범위	Week13 요구사항과 직접 관련 없는 이전 주차 기능, 일반 엔진 기능.

1. WEEK13의 구현 목표

WEEK13의 주제는 3차원 월드에서 보이는 초점과 보이지 않는 힘을 엔진 기능으로 연결하는 것이다. 과제 문서에서는 두 가지 DOF를 제시한다. 하나는 Depth of Field, 즉 카메라 초점과 흐림 효과이고, 다른 하나는 Degrees of Freedom, 즉 rigid body와 constraint의 운동 자유도이다.

따라서 이번 주 구현은 크게 렌더링 쪽의 Depth of Field와 물리 쪽의 PhysX 기반 rigid body, ragdoll, physics asset authoring 기능으로 나뉜다.

구분	과제에서 요구한 방향	프로젝트에서 확인되는 구현
Physics Core	PhysX SDK를 빌드하고 자체 엔진에 통합한다.	FPhysXPhysicsScene, FPhysXPhysicsRuntime, FPhysXBodyBuilder, FPhysXConstraintBuilder를 통해 PhysX scene/runtime/body/constraint 계층을 구현했다.
Physics Asset Editor	스켈레탈 메시의 물리 바디와 constraint를 편집하는 에디터를 구현한다.	UPhysicsAsset 데이터, FPhysicsAssetEditorWidget UI, 자동 바디 생성, validation, gizmo, preview mesh 렌더링으로 구성되어 있다.
Ragdoll Simulation	Body와 constraint를 생성하고 제한된 자유도로 ragdoll을 시뮬레이션한다.	FPhysicsAssetInstance가 asset 데이터를 runtime body/constraint handle로 바꾸고, USkeletalMeshComponent가 full/partial ragdoll pose를 skeletal pose에 반영한다.
Debug Render / Show Flag / Stat	Physic Body 렌더링, Show Flag, Stat을 지원한다.	FShowFlags, Physics debug snapshot, PhysicsAssetPreviewComponent, ShapeSceneProxy, FPhysicsStats, overlay stat UI가 연결되어 있다.
Vehicle	PhysX Vehicle API를 활용해 나만의 자동차를 구현한다.	FVehicleDesc/FVehicleHandle, FPhysXVehicleRuntime, UWheeledVehicleMovementComponent로 차량 생성, 입력, suspension raycast, vehicle update, wheel snapshot을 처리한다.
Depth of Field	focus distance/focus range 기반의 후처리 Depth of Field를 구현한다.	CoC 계산, background/foreground blur, bokeh scatter, composite, CoC debug view mode로 render pass가 분리되어 있다.

2. 전체 구조

이번 구현은 PhysX API를 엔진 코드 곳곳에서 직접 호출하는 방식이 아니라, 엔진용 물리 인터페이스와 runtime 계층을 사이에 두는 방식으로 되어 있다. 이렇게 하면 상위 게임 코드와 컴포넌트는 PhysX 타입을 거의 몰라도 되고, PhysX 관련 세부 구현은 Engine/Physics 아래에 모인다.

```
UWorld / Component
-> IPhysicsScene
  -> FPhysXPhysicsScene
    -> FPhysXPhysicsRuntime
      -> PxScene / PxRigidActor / PxShape / PxD6Joint / PxVehicleDrive4W
```

Physics Asset 쪽도 비슷하다. UPhysicsAsset은 에셋 데이터만 보관하고, 실제 시뮬레이션 중 만들어진 body와 constraint는 FPhysicsAssetInstance가 소유한다. 에디터는 UPhysicsAsset을 수정하고, 런타임은 이 에셋을 읽어서 PhysX body와 joint를 만든다.

```
UPhysicsAsset
-> BodySetups, ConstraintSetups
  -> FPhysicsAssetInstance
    -> FPhysicsBodyHandle, FPhysicsConstraintHandle
      -> PhysX rigid body, shape, D6 joint
```

3. PhysX Core 구현

PhysX 통합의 핵심 파일은 FPhysXPhysicsScene과 FPhysXPhysicsRuntime이다. FPhysXPhysicsScene은 엔진 월드가 소유하는 물리 scene adapter이고, FPhysXPhysicsRuntime은 실제 PhysX 객체의 생성, 파괴, 업데이트, snapshot 생성을 담당한다.

주요 파일	역할
Engine/Physics/IPhysicsScene.h	엔진 상위 코드가 사용하는 backend-neutral 물리 scene 인터페이스. body 등록, tick, query, force, vehicle API를 노출한다.
Engine/Physics/PhysicsRuntime.h	body/constraint 생성, force/impulse, transform/velocity, snapshot/debug/stat 수집을 추상화한다.
Engine/Physics/PhysXPhysicsScene.cpp	PxScene 생성, dispatcher/callback/filter shader 설정, physics thread 실행, frame submit/wait, event dispatch를 담당한다.
Engine/Physics/PhysXPhysicsRuntime.cpp	handle-generation 기반 body/constraint 관리, command queue, fixed timestep simulation, world snapshot publish를 담당한다.
Engine/Physics/PhysXBodyBuilder.cpp	엔진의 body/shape desc를 PxRigidActor와 PxShape로 변환한다. Box/Sphere/Capsule, trigger/query/simulation flag, filter data, mass/inertia를 적용한다.
Engine/Physics/PhysXConstraintBuilder.cpp	엔진 constraint desc를 PxJoint로 변환하고 linear/twist/swing motion 및 angular limit을 설정한다.
Engine/Physics/PhysXConversion.cpp	FVector, FQuat, FTransform과 PxVec3, PxQuat, PxTransform 사이의 변환을 담당한다.

3.1 Scene 초기화와 Tick 흐름

1. Initialize에서 PhysX Foundation/Physics를 확보하고 PxSceneDesc를 구성한다.
2. 중력은 엔진의 Z-up 기준에 맞춰 PxVec3(0, 0, -9.81f)로 설정한다.
3. PxDefaultCpuDispatcher, filter shader, simulation event callback, CCD/PCM/active actor/RW lock 옵션을 scene에 넣는다.
4. 게임 스레드는 SubmitPhysicsFrame으로 physics command를 넘기고, physics thread는 fixed dt/substep으로 simulate/fetchResults를 수행한다.
5. 시뮬레이션 결과는 FPhysicsWorldSnapshot으로 publish되고, 렌더링/게임 로직은 snapshot을 읽어서 transform을 갱신한다.

이 구조의 중요한 점은 PhysX 객체를 여러 스레드에서 직접 만지지 않도록 만드는 것이다. 게임 스레드는 명령을 queue로 넣고, physics thread가 안전한 타이밍에 명령을 적용한다. 이후 결과는 immutable snapshot으로 읽히기 때문에, 렌더링과 게임 코드가 물리 thread 내부 상태를 직접 건드리는 일을 줄인다.

3.2 Body, Shape, Constraint 생성

Body 생성은 FPhysXBodyBuilder가 담당한다. Static body는 PxRigidStatic, Dynamic/Kinematic body는 PxRigidDynamic으로 만들어진다. Shape는 Box, Sphere, Capsule을 지원한다. 특히 PhysX Capsule은 기본 축이 X축 기준인데 엔진은 Z축 capsule을 쓰기 때문에, capsule shape local rotation에서 축 보정을 수행한다.

Constraint 생성은 FPhysXConstraintBuilder가 담당한다. Week13의 핵심 개념인 Degrees of Freedom은 여기서 실제 코드로 반영된다. linear X/Y/Z, twist, swing1, swing2 축의 motion을 free, limited, locked로 설정하고, 제한이 있는 경우 twist limit과 swing cone limit을 PhysX D6 joint에 넣는다.

4. Physics Asset 데이터 구조

Physics Asset은 스켈레탈 메시의 본에 붙는 물리 바디와, 바디 사이의 constraint를 저장하는 에셋이다. Bone은 애니메이션 계층을 표현하지만 부피, 질량, 충돌, 감쇠, 관절 제한을 직접 표현하지 못한다. 그래서 별도의 BodySetup과 ConstraintSetup을 둔다.

구조체/클래스	저장하는 정보
FPhysicsAssetShapeSetup	shape 종류(Box/Sphere/Capsule), local transform, box half extent, sphere radius, capsule radius/half height.
FPhysicsAssetBodySetup	BoneName, BodyLocalFrame, shapes, mass, center of mass, damping, max angular velocity, CCD, gravity, solver iteration, linear/angular lock.
FPhysicsAssetConstraintSetup	ParentBoneName, ChildBoneName, parent/child local frame, linear/twist/swing limit, projection, body 간 collision disable 옵션.
UPhysicsAsset	BodySetups와 ConstraintSetups 배열, skeleton binding, asset path, add/update/remove/find/serialize 기능.

UPhysicsAsset은 실제 PhysX 객체를 직접 소유하지 않는다. 이 클래스는 저장 가능한 에셋 데이터이며, 런타임에서 이 데이터를 읽고 실제 body와 constraint를 만드는 쪽은 FPhysicsAssetInstance다.

5. Physics Asset Editor 구현

Physics Asset Editor는 UPhysicsAsset을 시각적으로 편집하기 위한 에디터 UI다. 코드상 핵심은 FPhysicsAssetEditorWidget이며, skeleton tree, constraint graph, body/shape/constraint details, validation panel, simulation controls가 이 위젯에 모여 있다.

기능	구현 내용
에셋 열기/저장	UPhysicsAsset을 편집 대상으로 받아 UI를 구성하고, SaveEditedPhysicsAsset에서 metadata를 보존한 채 저장한다.
Body/Shape 편집	선택된 bone의 body setup, shape setup을 details UI에서 수정한다. Box/Sphere/Capsule primitive를 다룬다.
Constraint 편집	parent/child bone, local frame, linear/twist/swing limit, body 간 collision disable 옵션을 편집한다.
자동 생성	FPhysicsAssetAutoBodyGenerator가 skeleton/mesh 정보를 기반으로 body를 생성한다. PCA 분석 또는 bone-axis 방식, primitive type, 작은 bone merge, helper bone skip, constraint 생성 옵션을 지원한다.
Validation	누락된 body, 잘못된 constraint, skeleton binding 문제 등을 검사하는 validation panel을 제공한다.
Debug JSON Export	asset path, preview mesh path, skeleton path, editor selection, graph layout, bodies, constraints를 JSON으로 내보낸다.
Editor Simulation	preview mesh component에 physics asset override를 걸고, 선택 body 또는 전체 body를 시뮬레이션해 확인한다.

5.1 Gizmo와 Preview 렌더링

에디터에서 body, shape, constraint frame을 직접 움직이기 위해 PhysicsAssetGizmoTarget이 사용된다. 이 대상 객체는 viewport gizmo transform을 asset의 local frame이나 shape 크기로 다시 환산한다. 예를 들어 shape scale delta가 들어오면 Box는 half extent, Sphere는 radius, Capsule은 radius와 half height로 변환된다.

보이는 미리보기는 PhysicsAssetPreviewComponent와 PhysicsAssetPreviewSceneProxy가 담당한다.

PreviewComponent는 현재 skeleton pose와 asset 데이터를 기반으로 Box/Sphere/Capsule mesh와 constraint limit surface를 만들고, SceneProxy는 이 데이터를 투명한 primitive로 렌더링한다. 즉 에디터의 Physics Asset 화면에서 반투명 body와 constraint limit을 볼 수 있게 하는 층이다.

6. Ragdoll Simulation 구현

Ragdoll의 흐름은 Physics Asset 데이터를 runtime body와 constraint로 바꾼 뒤, 시뮬레이션 결과를 다시 skeletal pose로 끌어오는 구조다. Week13 예시 코드가 PxRigidBodyDynamic body와 PxD6Joint를 만든 뒤 simulate/fetchResults 후 transform을 가져오는 형태라면, 프로젝트 구현은 이 흐름을 엔진 컴포넌트 구조에 맞게 일반화한 것이다.

- USkeletalMeshComponent가 사용할 UPhysicsAsset을 결정한다. component override, mesh default, skeleton default 순서로 찾는다.
- GetOrCreatePhysicsAssetInstance로 FPhysicsAssetInstance를 준비한다.

3. `FPhysicsAssetInstance::CreateBodiesAndConstraints`가 body mask를 계산하고 body setup마다 rigid body를 만든다.
4. constraint setup마다 parent/child body handle을 찾아 D6 constraint를 생성한다.
5. physics thread가 body를 시뮬레이션하고 world snapshot을 publish한다.
6. `USkeletalMeshComponent::ApplyPhysicsAssetPose`가 snapshot에서 bone transform을 재구성해 animation pose와 섞는다.

6.1 Full Body와 Partial Ragdoll

Full body ragdoll은 전체 body를 물리 시뮬레이션 대상으로 만들고, 스켈레탈 메시 pose를 거의 물리 결과에 맞춘다. 이때 독립 ragdoll collision 옵션, gravity, CCD, impulse 적용 등이 함께 설정된다.

Partial ragdoll은 특정 bone과 그 하위 body만 물리화한다. 예를 들어 상체, 왼팔, 오른팔, 머리/목 같은 preset을 기준으로 simulation body mask를 만들 수 있다. 선택 범위 밖의 bone은 animation pose를 유지하고, 경계 bone은 blend weight를 낮춰 갑작스러운 끊김을 줄인다.

Hit reaction도 구현되어 있다. 충격을 받은 bone을 기준으로 가장 적절한 simulated body를 찾고, impulse를 적용한다. 상태에 따라 partial ragdoll, partial refresh, full body ragdoll, impulse only, recovery 제한 등의 정책이 선택된다.

6.2 Pose Pull과 Recovery

시뮬레이션 결과를 가져올 때는 body world transform에 BodyLocalFrame inverse를 곱해 bone world transform을 계산한다. body가 없는 bone은 기존 animation pose나 parent 재구성을 사용해 유지한다. partial ragdoll에서는 적용 범위와 주변 bone에 다른 blend weight를 적용한다.

Ragdoll에서 다시 일어나는 흐름을 위해 recovery snapshot도 존재한다. pelvis와 chest의 방향을 기준으로 캐릭터가 엎어진 상태인지, 누운 상태인지 추정하고, 그에 맞는 stand-up animation을 선택할 수 있도록 되어 있다.

7. Debug Render, Show Flag, Stat

이번 주 요구사항에는 물리 body를 눈으로 확인할 수 있는 디버깅 렌더링과 Show Flag/Stat 지원이 포함되어 있다. 프로젝트에서는 이를 render options, show flag, debug snapshot, scene proxy, stat UI로 나누어 구현했다.

항목	구현 내용
Show Flag	FShowFlags에 bDoF, bCollision, bShowCollisionShape, bPhysicsAssetShapes, bPhysicsAssetConstraints, bPhysicsAssetBodySkeleton 등이 추가되어 렌더링 여부를 제어한다.
Collision Debug	ShapeSceneProxy와 DrawCommandBuilder가 collision/physics body debug geometry를 render command로 변환한다.
Physics Asset Preview	PhysicsAssetPreviewComponent가 asset body와 constraint surface를 mesh로 만들고, PhysicsAssetPreviewSceneProxy가 editor-only translucent object로 그린다.
Debug Snapshot	FPhysXPhysicsRuntime이 body/constraint debug snapshot을 publish하고, 렌더링 쪽은 PhysX 객체를 직접 읽지 않고 snapshot을 사용한다.

Physics Stat	FPhysicsStats가 body/shape/constraint 수, active dynamic body 수, contact/trigger pair, command 수, substep 수, simulate/fetch/snapshot/event dispatch 시간 등을 저장한다.
Stat UI	OverlayStatSystem과 EditorStatWidget이 수집된 stat을 에디터 화면에 보여주는 역할을 한다.

8. Vehicle 구현

Vehicle 구현은 PhysX Vehicle API를 엔진 추상화 뒤에 숨기는 방식으로 되어 있다. 상위 컴포넌트는 FVehicleDesc와 FVehicleInputState만 만들고, 실제 PxVehicleDrive4W 생성과 suspension raycast/update는 FPhysXVehicleRuntime이 담당한다.

주요 파일	역할
Engine/Physics/Vehicle/VehicleTypes.h	FVehicleHandle, FVehicleInputState, FVehicleWheelDesc, FVehicleDesc, FVehicleSnapshot 등 backend-neutral 차량 데이터 정의.
Engine/Physics/PhysXVehicleRuntime.cpp	PxVehicleDrive4W, wheel sim data, drive sim data, chassis actor, friction pair, batch query 생성 및 PreSimulate 처리.
Engine/Physics/PhysXVehicleInstance.h	PhysX vehicle, chassis actor, batch query, wheel query result, friction pairs, raw input을 한 차량 단위로 보관.
Engine/Component/Movement/WheeledVehicleMovementComponent.cpp	게임 입력을 받고 FVehicleDesc를 생성한다. 차량 생성/재생성, throttle/brake/steering/handbrake 입력, snapshot 소비, wheel debug를 담당한다.

차량 시뮬레이션 흐름은 다음과 같다.

1. UWheeledVehicleMovementComponent가 chassis, wheel setup, engine, transmission, tire, collision 설정으로 FVehicleDesc를 만든다.
2. IPhysicsScene::CreateVehicle을 통해 physics runtime에 차량 생성을 요청한다.
3. FPhysXVehicleRuntime은 PxVehicleWheelsSimData와 PxVehicleDriveSimData4W를 만들고 PxVehicleDrive4W를 setup한다.
4. 매 physics substep에서 입력 상태를 raw input으로 변환하고, suspension raycast를 수행한 뒤 PxVehicleUpdates를 호출한다.
5. 차량 chassis와 wheel 결과는 FVehicleSnapshot으로 publish되고, movement component가 이를 받아 visual component나 wheel pose에 반영한다.

9. Depth of Field 구현

Depth of Field는 후처리 render pass 체인으로 구현되어 있다. 핵심은 depth를 이용해 CoC, 즉 circle of confusion 값을 계산하고, 초점 거리에서 떨어진 픽셀일수록 더 강한 blur를 적용하는 것이다. Week13 과제에서 제시된 blurAmount 식은 프로젝트에서 별도의 DoF setup pass와 composite pass로 확장되어 있다.

구성 요소	구현 내용
-------	-------

Render Options	FViewportRenderOptions에 DoFFocusDistance, DoFFocusRange, DoFMaxBlurRadius, DoFBokehRadiusThreshold, DoFBokehLumaThreshold, DoFBokehIntensity가 있다.
Show Flag / View Mode	FShowFlags::bDoF로 DoF 활성화를 제어하고, EViewMode::DoFCoC로 CoC debug 화면을 볼 수 있다.
Render Targets	Viewport가 CoCTexture, DoFBackgroundTexture, DoFForegroundTexture, DoFBokehTexture를 생성한다. CoC는 R16_FLOAT, bokeh는 half-resolution R16G16B16A16_FLOAT 계열을 사용한다.
DoFSetupPass	SceneDepth를 읽어 CoC texture를 만든다.
Background/Foreground Blur	SceneColor, SceneDepth, CoC를 읽어 초점 뒤쪽과 앞쪽 blur를 분리해서 계산한다.
Bokeh Scatter	밝기와 blur 반경 조건을 만족하는 픽셀을 downsampled bokeh target에 흘려보낸다.
Composite	원본 SceneColor, background blur, foreground blur, bokeh texture를 합성한다. CoC debug mode에서는 blur 결과 대신 CoC 시각화 shader를 사용한다.
Lua/Camera Binding	APlayerCameraManager의 Depth of Field와 Bokeh 설정을 Lua에 바인딩해 런타임에서 값 변경이 가능하게 했다.

10. 구현 흐름 요약

Week13 구현을 한 문장으로 정리하면, 물리 데이터 authoring부터 PhysX runtime simulation, debug visualization, 렌더링 후처리까지 한 주제 안에서 연결한 것이다. Physics Asset Editor에서 body와 constraint를 만들고 저장하면, 런타임의 FPhysicsAssetInstance가 같은 데이터를 사용해 PhysX rigid body와 D6 joint를 만든다. 시뮬레이션 결과는 snapshot으로 publish되고, SkeletalMeshComponent가 이를 bone pose로 되돌려 ragdoll을 보여준다.

Depth of Field는 물리 시스템과 직접 연결되지는 않지만, Week13의 다른 DOF인 camera focus를 담당한다. Show Flag와 View Mode를 통해 effect를 켜고 끌 수 있고, CoC debug 화면으로 흐름 기준을 확인할 수 있다.

11. 주요 파일 맵

영역	파일
Physics Core	Engine/Physics/IPhysicsScene.h Engine/Physics/PhysicsRuntime.h Engine/Physics/PhysXPhysicsScene.cpp Engine/Physics/PhysXPhysicsRuntime.cpp Engine/Physics/PhysXBodyBuilder.cpp Engine/Physics/PhysXConstraintBuilder.cpp Engine/Physics/PhysXConversion.cpp
Physics Asset Data	Engine/Physics/PhysicsAsset.h Engine/Physics/PhysicsAsset.cpp Engine/Physics/PhysicsAssetTypes.h

Physics Asset Runtime	Engine/Physics/PhysicsAssetInstance.h Engine/Physics/PhysicsAssetInstance.cpp Engine/Component/Primitive/SkeletalMeshComponent.cpp
Physics Asset Editor	Editor/UI/Asset/Physics/PhysicsAssetEditorWidget.h Editor/UI/Asset/Physics/PhysicsAssetEditorWidget.cpp Engine/Physics/PhysicsAssetAutoBodyGenerator.cpp Engine/Gizmo/PhysicsAssetGizmoTarget.cpp
Preview / Debug Render	Engine/Component/Debug/PhysicsAssetPreviewComponent.cpp Engine/Render/Proxy/PhysicsAssetPreviewSceneProxy.cpp Engine/Render/Command/DrawCommandBuilder.cpp Engine/Render/Types/ViewTypes.h
Stats	Engine/Physics/PhysicsStats.h Editor/Subsystem/OverlayStatSystem.cpp Editor/UI/Panel/EditorStatWidget.cpp
Vehicle	Engine/Physics/Vehicle/VehicleTypes.h Engine/Physics/PhysXVehicleRuntime.cpp Engine/Physics/PhysXVehicleInstance.h Engine/Component/Movement/WheeledVehicleMovementComponent.cpp
Depth of Field	Engine/Render/RenderPass/DoFSetupPass.cpp Engine/Render/RenderPass/DoFBackgroundBlurPass.cpp Engine/Render/RenderPass/DoFForegroundBlurPass.cpp Engine/Render/RenderPass/DoFBokehScatterPass.cpp Engine/Render/RenderPass/DoFPass.cpp Engine/Viewport/Viewport.cpp Engine/Render/Shader/ShaderManager.cpp

12. 현재 구현의 의미와 남은 개선점

현재 구현은 단순히 PhysX 예제 코드를 붙인 수준이 아니라, 엔진의 에셋, 컴포넌트, 렌더링, 디버그 UI 구조에 물리 시스템을 연결한 형태다. 특히 UPhysicsAsset과 FPhysicsAssetInstance를 분리한 점은 Unreal의 PhysicsAsset/BodyInstance 구조와 비슷한 방향이다. 저장 가능한 authoring data와 실행 중 mutable simulation state를 분리했기 때문이다.

남은 개선점은 다음과 같이 정리할 수 있다.

- Physics Asset Editor에서 constraint limit surface와 실제 PhysX limit 사이의 시각적 일치도를 더 검증해야 한다.
- PVD 연결이나 physics event inspection 도구를 추가하면 constraint break, contact, trigger 디버깅이 쉬워진다.
- Ragdoll recovery는 stand-up animation과 gameplay state machine까지 연결되면 완성도가 올라간다.

- Partial ragdoll은 animation blend boundary가 어색해질 수 있으므로 hit bone별 preset과 weight curve 조정이 필요하다.
- Vehicle은 타이어 마찰, suspension, chassis center of mass 튜닝 UI와 wheel visual sync 검증이 중요하다.
- Depth of Field는 현재 focus distance/range 기반 구현이므로, 실제 F-stop, focal length, sensor size에 가까운 물리 기반 카메라 모델로 확장할 수 있다.
- Show Flag와 Stat은 기능이 늘수록 항목이 많아지므로 category별 접기/필터링과 frame history 그래프가 있으면 분석이 쉬워진다.

13. 결론

Week13 파트의 핵심은 두 가지 DOF를 엔진 안에서 구현하는 것이었다. Degrees of Freedom 쪽에서는 PhysX rigid body, shape, D6 joint, Physics Asset, ragdoll, vehicle, debug/stat 체계를 만들었다. Depth of Field 쪽에서는 CoC 계산부터 blur, bokeh, composite, debug view까지 이어지는 후처리 파이프라인을 만들었다.

따라서 이번 주 구현은 “스켈레탈 메시의 물리 바디를 에디터에서 만들고, 런타임에서 PhysX로 시뮬레이션하고, 그 결과를 다시 화면과 스켈레톤 pose에 반영하는 흐름”과 “카메라 초점 기반 후처리 흐름을 렌더 파이프라인에 넣는 흐름”으로 정리할 수 있다.

참고 기준

- WEEK13 과제 문서: Physics Asset Editor, RagDoll Simulation, Show Flag/Stat, Debug Physic Body Rendering, Vehicle, Depth of Field, PhysX SDK 통합 항목.
- 분석 프로젝트: Jungle_Week13_Team2.zip 내부 KRAFTONEngine/Source의 Week13 관련 구현 파일.